

人工智能 Python 编程

Python 进阶基础

Chengwei QIN @ AI Thrust

类与对象

- OOP、构造/析构函数、类/对象级属性、魔术方法

继承

- 子类/父类、多态

模块与包

- 内置/用户自定义模块、import、`__name__ == '__main__'`、包

进阶迭代：递归与生成器

- yield 语句



类与对象

面向对象编程

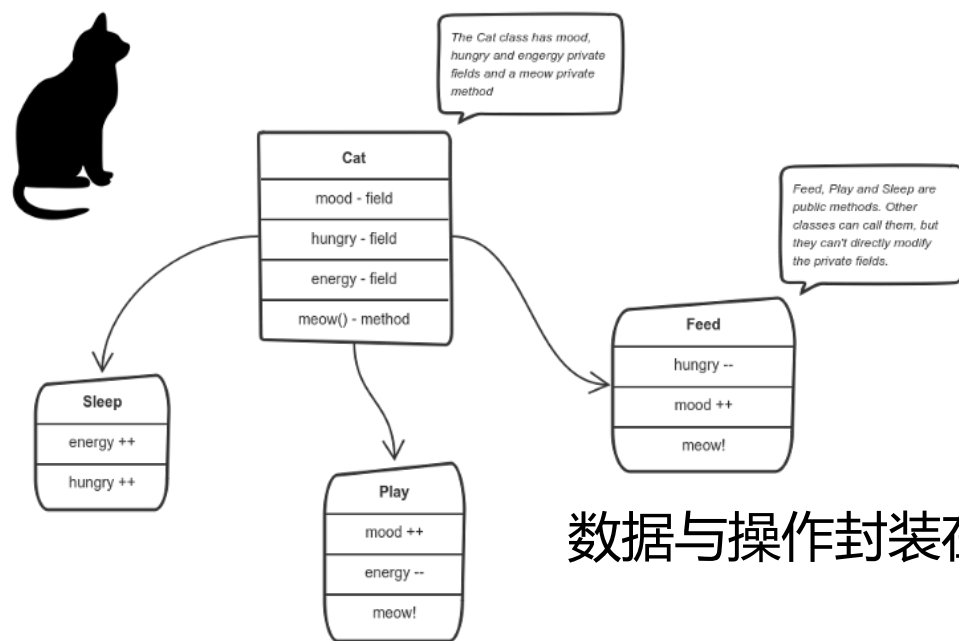
编程的两大范式：

- 过程式编程
- 面向对象编程 (OOP)

OOP 的核心思想：将数据与操作封装在对象中，并把程序设计为一组相互作用的对象。

OOP 示例：

- 有一个名叫 Alice 的人
- *Alice 收养了一只猫 Tom*
- *Tom 因饥饿而喵叫*
- *Alice 买了一条鱼*
- *Alice 用鱼喂了 Tom*
- *Tom 睡了 10 分钟*



数据与操作封装在猫对象中

■ 类与对象

类定义对象的抽象特征，规定其属性与方法

- **属性：**表示该类对象所具有的性质或数据
- **方法：**定义该类对象可执行的行为（可涉及其他对象、依赖属性值、修改属性）

Human

- 属性： name、sanity、health、age、money
- 方法： 领养宠物、喂猫、买食物、吃、玩、开车

Cat

- 属性： 饥饿度、口渴度、能量、快乐值
- 方法： meow、eat、sleep

Fish

- 属性： size、type、price
- 方法： swim

OOP 示例:

- 有一个名叫 Alice 的人
- Alice 收养了一只猫 Tom
- Tom 因饥饿而喵叫
- Alice 买了一条鱼
- Alice 用这条鱼喂了 Tom
- Tom 睡了 10 分钟

类与对象

对象是类的一个独立实例

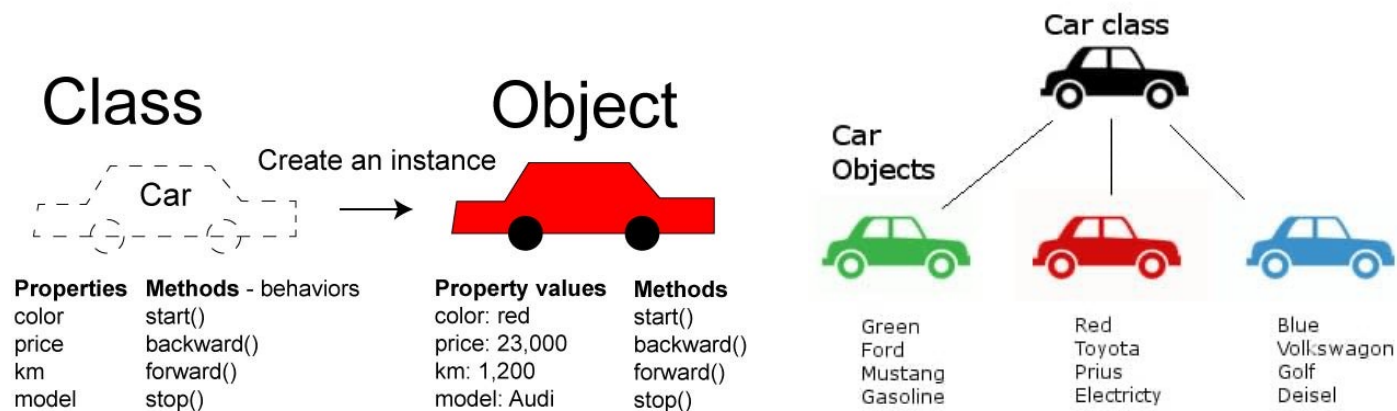
- 类好比建造某物的蓝图，而实例（也称对象）则是按蓝图造出的实物。
- 每个对象各自存储其属性的具体数据
- 每个对象都可以调用其类中定义的方法

类:

- Human
- Cat
- Fish

对象:

- Alice
- Tom
- Tom 吃掉的那条鱼

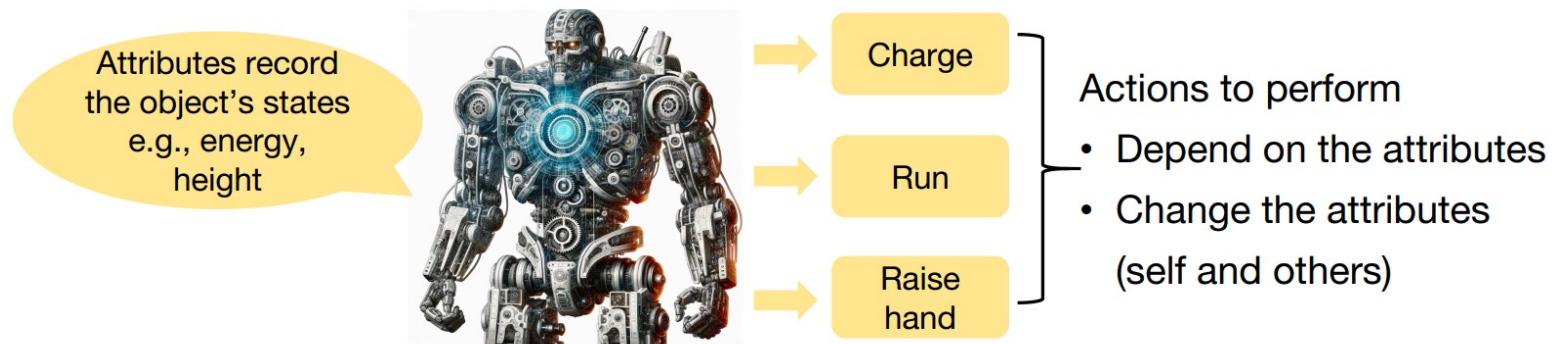


OOP 示例:

- 有一个名叫 Alice 的人
- *Alice 收养了一只猫 Tom*
- *Tom 因饥饿而喵叫*
- *Alice 买了一条鱼*
- *Alice 用这条鱼喂了 Tom*
- *Tom 睡了 10 分钟*

OOP 的基本原则：将数据（属性）与函数（方法）打包在对象内部。

- **模块化：使复杂程序结构清晰、更易理解**
- **可复用性：隐藏实现细节，使对象可作为黑盒复用**
- **安全性：限制直接访问以保护内部状态，降低误用风险**



在 Python 中，一切皆对象，例如：数据类型、函数、异常

- 内置类型如 list、tuple、string、dictionary、set、number、NoneType、Boolean、...
- 每个函数都是对象
- 每次 raise 的异常都是对象
- 以及更多

简单的类定义

语法:

```
01. class ClassName:
02.     # Statements to define attributes
03.     # Statements to define methods
```

```
class MyClass:
    x = 5
✓ 0.0s
```

创建对象

类名当作函数调用即可创建对象

- 类中定义的属性由所有对象共享

定义类时若尚未指定属性或方法，可用“pass”语句作为占位

```
p1 = MyClass()
print(p1.x)
✓ 0.0s
5
```

```
24 class AITrust:
25     pass
```

类方法与对象方法

类方法作用于类本身，对象方法作用于由该类创建的某个具体对象。

```
class Student:
    school_name = "Peking U"

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name

    #object method
    def change_sch(self, new_name):
        self.school_name = new_name
        return
```

```
# Object method only changes the object
Mary = Student()
Mary.change_sch("HKUST(GZ)")
Mary.school_name
```

'HKUST(GZ)'

```
# The class attribute is not changed
Jack = Student()
Jack.school_name
```

'Peking U'

```
# class method changes the class
Student.change_school("HKUST(GZ)")
Jack = Student()
Jack.school_name
```

'HKUST(GZ)'

类方法定义

语法（与定义普通函数类似）

```
@classmethod
def function_name(cls, param1, param2, ...):
    # put method statements here
```

- 第一个参数：表示类本身，用于访问其属性与方法
 - 名称可以任意，但约定使用 cls
 - 调用方法时会自动、隐式传入

```
# Class method
class Student:
    # Class attribute shared by all instances
    school_name = "High School"

    #Class Method
    @classmethod
    def change_school(cls, new_name):
        # Uses 'cls' to modify the shared class attribute
        cls.school_name = new_name
```

对象方法定义

语法（与定义普通函数类似）

```
def function_name(self, param1, param2):  
    # put method statements here
```

- 第一个参数：表示对象本身，用于访问其属性与方法
 - 名称可以任意，但约定使用 self
 - 调用方法时会自动、隐式传入

```
class Human:  
    age = 10  
    def print_age(self):  
        print(self.age)
```

```
student_1 = Human()  
student_1.print_age()
```

对象 student_1 会自动传入方法
student_1.print_age()

任何用户自定义类都是如下类的子类 `object`

- `object` 是 Python 的内置类。Python 中的任何类（包括用户自定义类）都是它的子类。

```
[5]: class Person:
      pass

      issubclass(Person, object)
```

- Person 继承

的属性与 `object` 的属性和方法。

`object`

```
[12]: dir(object)
```

```
[12]: ['__class__',
      '__delattr__',
      '__dir__',
      '__doc__',
      '__eq__',
      '__format__',
      '__ge__',
      '__getattribute__']
```

```
[11]: class Person:
      pass
```

```
      dir(Person)
```

```
[11]: ['__class__',
      '__delattr__',
      '__dict__',
      '__dir__',
      '__doc__',
      '__eq__',
```

- 我们用 `object` 表示 Python 的内置类。
- 我们用 `object` 指代类的实例。

构造函数 `__new__()`

`__new__()`

- 任何类都从如下类继承该方法 `object`
- 创建实例时自动调用（为该类分配内存）。

```
[16]: class Person:  
        pass  
  
Mike = Person() #__new__ is called and the instance Mike is created
```

■ 初始化方法 `__init__`

`__init__()`: 创建对象时会自动、隐式调用。

- 默认的 `__init__()` 是直接从 `object` 继承而来的那个。
- 虽然 `__init__` 实际是初始化方法而非构造函数，但常被称为构造函数。

```
•[16]: class Person:
        pass

Mike = Person() #__new__() is called and the instance Mike is created;
                #__init__() is called and it does nothing here since __init__() is not overloaded when defining Person;
```

■ 重写 `__init__()`

可以重新定义（重写）`__init__()`，为已创建的对象添加属性。

```
[26]: class Person:
      age = 10
      def __init__(self, wt):
          self.weight = wt

Jack = Person(wt=100) #1. __new__ allocates memory and returns a brand-new instance of Person.
                        #2. __init__ creates Jack.weight and assigns 100 to it
print("Jack age:", Jack.age) # Python looks for age first from Jack (fails), and then from Person
print("Jack weight:", Jack.weight)

Jack age: 10
Jack weight: 100
```

方法

属性可在方法调用过程中动态创建

```
class Human:
    age = 10

    def set_name(self, name_x):
        self.name = name_x
```

```
student_1 = Human()
print(student_1.name)
```

⊗ ✨ 0.0s

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[9], line 2
      1 student_1 = Human()
----> 2 print(student_1.name)
```

AttributeError: 'Human' object has no attribute 'name'

```
student_1.set_name('John')
print(student_1.name)
```

John

属性

Python 类中的两类属性

- 类级属性：由该类的所有对象共享
- 对象级属性：为每个对象单独创建，常在 `__init__()` 等方法中定义

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

类级

对象级

属性

两类属性

- **类级属性：由所有对象共享**
- **对象级属性：为每个对象单独创建，常在 `__init__()` 等方法中定义**

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

类级

对象级

```
alice = Human('Alice', 25)
bob = Human('Bob', 30)
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Guangzhou Guangzhou
4345001776 4345001776

```
Human.location = 'Hong Kong'
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Hong Kong Hong Kong
4836193328 4836193328

一次性修改所有对象的类级属性

```
alice.location = 'Hong Kong'
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Hong Kong Guangzhou
4836171440 4345001776

仅为一个对象修改类级属性不会影响其他对象

属性

两类属性

- **类级属性：由所有对象共享**
- **对象级属性：为每个对象单独创建，常在 `__init__()` 等方法中定义**

特殊情况：可变类属性

- 若类属性引用可变对象（如 list 或 dictionary），通过一个实例所做的修改会影响所有共享它的实例

```
class Human:
    location = 'Guangzhou'
    lst = []
    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=20)
print(alice.lst, bob.lst)
print(id(alice.lst), id(bob.lst))
alice.lst.append(1)
print(alice.lst, bob.lst)
```

✓ 0.0s

```
[] []
4836004224 4836004224
[1] [1]
```

属性

两类属性

- 类级属性：由所有对象共享
- 对象级属性：为每个对象单独创建，常在 `__init__()` 等方法中定义

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

类级

对象级

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=23)
print(alice.age, bob.age)
print(id(alice.age), id(bob.age))
```

✓ 0.0s

22 23
4344769488 4344769520

初始化时创建

```
alice.height = 162
print(alice.height)
```

✓ 0.0s

Python

162

手动创建

```
print(bob.height)
```

✗ 0.0s

Python

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[43], line 1
----> 1 print(bob.height)

AttributeError: 'Human' object has no attribute 'height'
```

互不影响

删除

删除对象的某个属性，或删除整个对象。

- 使用 del 关键字

```
1 Joe = Person('Joe', 18, "HKUST(GZ)-5B")
2 print(Joe.age)
3 del Joe.age
4 print(Joe.age)
```

[11] 0.0s Python

... 18

... -----

AttributeError Traceback (most recent call last)

Cell In[11], line 4

2 print(Joe.age)

3 del Joe.age

----> 4 print(Joe.age)

AttributeError: 'Person' object has no attribute 'age'

删除属性

```
1 Joe = Person('Joe', 18, "HKUST(GZ)-5B")
2 print(Joe.age)
3 del Joe
4 print(Joe.age)
```

[12] 0.0s Python

... 18

... -----

NameError Traceback (most recent call last)

Cell In[12], line 4

2 print(Joe.age)

3 del Joe

----> 4 print(Joe.age)

NameError: name 'Joe' is not defined

删除对象

析构函数

`__del__()`: 对象销毁时自动调用

- 默认析构函数: 默认不执行任何操作
- 自定义析构函数: 定义对象销毁时要执行的特定操作

自动垃圾回收: 当对象的所有引用都被删除时, 该对象被销毁

```
1 class Human:
2     _location = 'Guangzhou'
3
4     def introduce(self):
5         print("I am {} from {}. I am {} years old. "\
6             .format(self.name, self.location,
7                 self.age))
8
9     def __init__(self, name, age):
10        self.name = name
11        self.age = age
12        print('object created')
13
14    def __del__(self):
15        print('object deleted')
```

```
17  alice = Human(name='Alice', age=22)
18  del alice
19  print('hello')
```

```
object created
object deleted
hello
```

双下划线方法

双下划线（**Dunder**）方法是 **Python** 在使用内置功能或符号时自动运行的特殊方法（即无需显式调用）。

- `__new__`: 构造函数
- `__init__`: 初始化方法
- `__del__`: 析构函数
- `__lt__`、`__gt__`、`__eq__`、`__ge__`、`__le__`: 比较运算 `<`、`>`、`==`、`<=`、`>=`
- `__add__`、`__sub__`、`__mul__`、`__truediv__`、`__mod__`: 算术运算
- `__str__`: 定义对象的字符串表示（供 `str()` 使用）
- `__getitem__`: 支持用 `[i]` 进行索引
- `__len__`: 定义 `len(obj)` 的行为
- `__getattr__`: 访问未定义属性时调用
- `__setattr__`: 设置属性时调用
- `__delattr__`: 删除属性时调用

■ 双下划线方法示例

- 使用符号 (+) 时: 写 `a + b` 会自动触发 `a.__add__(b)`。
- 使用内置函数 (`len()`) 时: 写 `len(my_list)` 会自动触发 `my_list.__len__()`。
- 创建对象 (`Person()`) 时: 写 `p = Person()` 会自动触发 `Person.__new__()` 和 `Person.__init__()`。

重载 __str__

`__str__()` 方法定义于 `object`。因此，每个用 `object` 的类都从 `object` 继承该方法。
当对象传入 `print()` 时，`print` 会调用其 `__str__()` 方法以获得可读的字符串表示。

```
class Human:
    location = 'Guangzhou'
    lst = []
    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return "Name:{} Location: {} Age: {}"\
               .format(self.name, self.location, self.age)
```

```
alice = Human(name='Alice', age=22)
print(alice)
```

✓ 0.0s

<__main__.Human object at 0x106808100>

重载前：默认字符串

```
alice = Human(name='Alice', age=22)
print(alice)
```

✓ 0.0s

Name:Alice Location: Guangzhou Age: 22

重载后

用 `__lt__()` 方法重载 '`<`' 运算符

`def __lt__(self, other)`

- 比较两个对象：若 `self < other` 则返回 `True`，否则返回 `False`

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __lt__(self, other):
        return self.age < other.age
```

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=20)
print(alice < bob)
```

⊗ 0.0s

未重载时

```
TypeError                                Traceback (most recent call last)
Cell In[60], line 3
      1 alice = Human(name='Alice', age=22)
      2 bob = Human(name='Bob', age=20)
----> 3 print(alice < bob)

TypeError: '<' not supported between instances of 'Human' and 'Human'
```

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=25)
print(alice < bob)
```

✓ 0.0s

重载后

Python

True



`__lt__()` 方法定义于 `object` 类中，每个用户定义的类都继承默认的 `__lt__()` 方法。

使用 `<` 运算符比较两个对象时，Python 会尝试调用 `__lt__()` 进行比较。

默认方法未为 `Human` 对象提供比较规则。因此需要重写 `__lt__()`，定义对本类对象使用 `<` 的含义。本例中比较 `self.age` 与 `other.age`

用 `__len__()` 方法重载内置函数 `len()`

`def __len__(self)`

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(len(team))
```

⊗ ✨ 0.0s

未重载时

TypeError Traceback (most recent call last)

Cell In[75], line 4

```
2 team.add_member('Alice')
3 team.add_member('Bob')
----> 4 print(len(team))
```

TypeError: object of type 'Team' has no len()

```
class Team:
    def __init__(self):
        self.members = []

    def add_member(self, name):
        self.members.append(name)

    def __len__(self):
        return len(self.members)
```

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(len(team))
```

✓ 0.0s

2

重载后

__getitem__

用 `__getitem__()` 方法重载索引操作 `[i]`

`def __getitem__(self, i):`

- 接收索引作为参数，并返回对应元素

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(team[1])
```

⊗ ✨ 0.0s

未重载时

TypeError Traceback (most recent call last)

Cell In[79], line 4

```
2 team.add_member('Alice')
3 team.add_member('Bob')
----> 4 print(team[1])
```

TypeError: 'Team' object is not subscriptable

```
class Team:
    def __init__(self):
        self.members = []

    def add_member(self, name):
        self.members.append(name)

    def __len__(self):
        return len(self.members)

    def __getitem__(self, i):
        return self.members[i]
```

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(team[1])
```

✓ 0.0s

重载后

Bob

任务：构建一个简单的图书管理系统，支持添加、借出和归还图书。

Book 类

- 属性：ID、title、author，以及表示图书是否已借出的状态
- 方法：
 - 构造函数：创建一本新书
 - borrow_book(): 标记为已借出
 - return_book(): 标记为未借出
 - get_status(): 返回 "Available" 或 "Borrowed"

Library 类

- 属性：
 - Book 对象的集合
- 方法：
 - add_book(book): 向目录添加一本新书。
 - borrow_book(id): 借书；若已借出，则打印 "Not available" 。
 - return_book(id): 归还图书。
 - list_available_books(): 按 id 升序打印所有可借图书的信息
 - 支持索引：Library[i] 返回第 i 本加入图书馆的书



继承

继承是一种机制，使新类（子类）能够获得已有类（父类或基类）的属性和方法

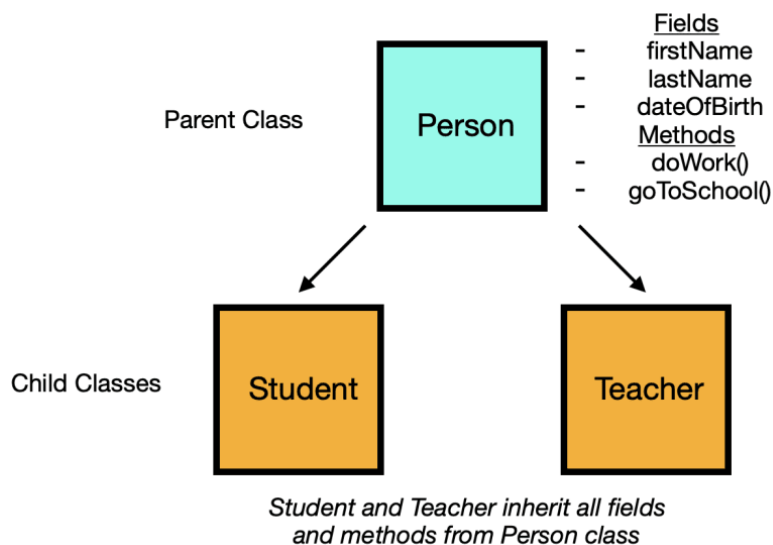
- 以层次化方式组织与结构化代码

特点

- **可复用性**
 - 子类可直接复用父类代码
- **可扩展性**
 - 子类可定义新的属性和方法
 - 可重写父类方法以提供更具体的功能

示例:

- 人具有 names、location 等属性, 以及 eat()、drink() 等方法
- 'teachers' 和 'students' 都是 'person' 的子类, 因此继承这些属性和方法
- 子类可以扩展功能



父类 (超类)

- 被继承的类

子类

- 从父类继承的类

语法: `class SubClassName(ParentClassName)`

子类继承其父类的全部属性和方法

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am a human from {}".format(self.location))
```

```
class 子类Student(父类Human):
    def __init__(self, id):
        self.id = id
stu = Student(id='HKUST(GZ)-456')
stu.introduce()
print(stu.id)
```

✓ 0.0s

```
I am a human from Guangzhou.
HKUST(GZ)-456
```

子类不会自动调用超类的初始化方法

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
        print('Human constructed')
```

```
class Student(Human):
    def __init__(self, id):
        self.id = id
stu = Student(id='HKUST(GZ)-456')
stu.introduce()
print(stu.id)
```

⊗ 0.0s

AttributeError Traceback (most recent call last)

Cell In[95], line 5

```
3         self.id = id
4 stu = Student(id='HKUST(GZ)-456')
----> 5 stu.introduce()
6 print(stu.id)
```

Cell In[94], line 6

```
4 def introduce(self):
5     print("I am {} from {}. I am {} years old. "\
----> 6         .format(self.name, self.location,
7                 self.age))
```

AttributeError: 'Student' object has no attribute 'name'

显式调用超类的初始化方法

SuperClass.__init__(self, arguments)

- 必须用 self 显式传入当前对象

```
class Student(Human):  
    def __init__(self, name, age, id):  
        Human.__init__(self, name, age)  
        self.id = id  
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')  
alice.introduce()  
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed  
I am Alice from Guangzhou. I am 22 years old.  
Alice 22 HKUST(GZ)-456
```

显式调用超类的初始化方法

super(ClassName, object).__init__(arguments)

- super 对象：返回一个特殊对象，用于访问超类（ClassName 的超类）的函数
- *super()*：在 Python 3 中等价于 *super(ClassName, self)*

```
class Student(Human):
    credits = 0
    def __init__(self, name, age, id):
        super(Student, self).__init__(name, age)
        self.id = id
alice = Student('Alice', 22, id='HKUST(GZ)-456')
alice.introduce()
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed
I am Alice from Guangzhou. I am 22 years old.
Alice 22 HKUST(GZ)-456
```

```
class Student(Human):
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
alice.introduce()
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed
I am Alice from Guangzhou. I am 22 years old.
Alice 22 HKUST(GZ)-456
```

子类可在继承超类的基础上定义额外的属性和方法

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
class Student(Human):
    credits = 0

    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def gain_credit(self, credits):
        self.credits += credits

    def print_credit(self):
        print("I have {} credits".format(self.credits))

alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
alice.introduce()
alice.gain_credit(10)
alice.print_credit()
```

子类可以重写超类中定义的方法

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

✓ 0.0s

调用优先级

- 默认调用当前类的方法
- 若当前类没有该方法，则调用超类的方法

```
class Student(Human):
    credits = 0

    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am a student from HKUST-GZ. My name is {}. I am {} years old. "\
              .format(self.name, self.age))

    def gain_credit(self, credits):
        self.credits += credits

    def print_credit(self):
        print("I have {} credits".format(self.credits))
```

```
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
```

```
alice.introduce()
```

```
super(Student, alice).introduce()
```

✓ 0.0s

I am a student from HKUST-GZ. My name is Alice. I am 22 years old.

I am Alice from Guangzhou. I am 22 years old.

- 多态指不同对象以各自的方式实现同名方法。
- 面向对象的重要特性，常建立在继承之上

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

✓ 0.0s

```
class Student(Human):
    credits = 0
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am a student from HKUST-GZ. My name is {}. I am {} years old. "\
              .format(self.name, self.age))
```

```
class Professor(Human):
    credits = 0
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am {}, a professor in HKUST-GZ.".format(self.name))
```

```
alice = Student('Alice', 20, 'HKUST(GZ)-456')
alice.introduce()
bob = Professor('Bob', 40, 'GZ-0000123')
bob.introduce()
```

✓ 0.0s

I am a student from HKUST-GZ. My name is Alice. I am 20 years old.
I am Bob, a professor in HKUST-GZ.

任务：创建基类 Animal，并派生 Bird 和 Mammal 两个类。各类的属性和方法用于演示继承概念

基类 - Animal

- 属性: **name (str)**、**age (int)**
- 方法:
 - `__init__(self, name, age)`
 - `make_sound(self)`: 打印通用叫声。

派生类

- **Bird**
 - 特有属性: **wing_span** —— 翼展 (米)。
 - 方法重写:
 - `__init__(self, name, age, wing_span)`
 - `make_sound(self)`: `print "Chirp"`。
- **Mammal**
 - 特有属性: **fur_color (str)** —— 毛发颜色。
 - 方法重写:
 - `__init__(self, name, age, fur_color)`
 - `make_sound(self)`: `print "Roar"`。



模块与包

什么是模块

包含 Python 定义（如函数、类）和语句的文件，以 .py 结尾
文件名即为模块名

动机：

- 组织：将相关函数与类集中管理。
- 复用：跨项目使用同一份代码，无需复制粘贴
- 命名空间：避免代码不同部分的名称冲突。`my_module.my_func()` 不会与 `other_module.my_func()` 冲突
- 内置模块：`math`、`string`、`date`、`random` 等
- 自定义模块：将源码保存为扩展名为 `.py` 的文件即可

使用模块: import 语句

Syntax	Notes
<code>import module</code>	Best practice. Access via <code>module.name</code> .
<code>import module as m</code>	Good for long names (e.g., <code>numpy as np</code>).
<code>from module import a, b</code>	Brings specific names into your scope.
<code>from module import *</code>	Avoid! Can cause name collisions.

```
1 # import the whole module
2 import math
3 print(math.pi)
4 # import selected names
5 from math import sqrt, sin
6 print(sqrt(9)) # Don't need math.sqrt()
7 # alias a module
8 import numpy as np
9 arr = np.array([1, 2, 3])
```

自定义模块示例

创建自己的模块来存放 AI 工具函数。

File: ai_utils.py

```
1 # ai_utils.py
2 import numpy as np
3 def softmax(x):
4     """Compute softmax
5     values for each
6     sets of scores in x
7     ."""
8     e_x = np.exp(x - np.
9         max(x))
10    return e_x / e_x.sum()
11 VERSION = "1.0"
```

File: main.py (in same folder)

```
1 # main.py
2 import ai_utils
3 import numpy as np
4 logits = np.array([2.0,
5     1.0, 0.1])
6 probabilities = ai_utils.
7     softmax(logits)
8 print(f"Probs: {
9     probabilities}")
10 print(f"Using utils v{
11     ai_utils.VERSION}")
```

结果：已成功分离逻辑。main.py 负责任务编排，ai_utils.py 提供可复用组件

■ `__name__ == '__main__'`

- Python 为每个模块设置特殊变量 `__name__`
- 若文件被直接运行（如 `python my_module.py`），其 `__name__` 为 `"__main__"`。
- 若文件被导入，其 `__name__` 为模块名（如 `"my_module"`）
- 据此可编写仅在直接运行时执行的代码。这是为模块加入测试或示例的标准做法

```
1 # in ai_utils.py
2 def softmax(x):
3     # ... (implementation from before) ...
4 # This block only runs when you execute "python ai_utils.py"
5 if __name__ == "__main__":
6     print("Running a test of the softmax function...")
7     test_data = np.array([3.0, 1.0, 0.2])
8     test_result = softmax(test_data)
9     print(f"Test input: {test_data}")
10    print(f"Test output: {test_result}")
11    assert np.isclose(np.sum(test_result), 1.0)
12    print("Test passed!")
```

从模块到包

- 项目变大后，仅靠若干 .py 文件可能不够组织。
- Python 包就是存放模块的目录。
- AI 项目的常见结构示例如下：
 - `__init__.py` 可以为空。
 - 它们告知 Python 该目录是一个包。

```
my_ai_project/  
|-- main.py  
|-- data/  
|   |-- __init__.py  
|   |-- loaders.py  
|-- models/  
|   |-- __init__.py  
|   |-- cnn.py  
|-- utils/  
    |-- __init__.py  
    |-- metrics.py
```

从包中导入

- 在包结构中，使用点号导入。
假设正在运行 main.py:

```
1  # in main.py
2  # Absolute imports from the project root
3  from data.loaders import load_image_data
4  from models.cnn import SimpleCNN
5  from utils.metrics import calculate_f1_score
6  # --- Or using aliases ---
7  import utils.metrics as metrics
8  # ... code that uses these imported functions ...
9  model = SimpleCNN()
10 images = load_image_data('path/to/images')
11 # ...
12 f1 = metrics.calculate_f1_score(preds, labels)
```

该结构可扩展，scikit-learn、tensorflow、pytorch 等主流 AI 库均采用。

任务：通过创建模块并组织为包，快速搭建 Python 项目结构。

1. 创建并使用模块：

- 编写含 `normalize()` 函数的 `data_utils.py`。
- 编写 `main.py`，导入并调用该函数。

2. 添加测试保护：

- 在 `data_utils.py` 中加入 `if __name__ == "__main__":` 块，自测 `normalize()`。

3. 构建包并修正导入：

- 按下方包结构重组文件（创建文件夹和空的 `__init__.py`）。
- 然后修正 `main.py` 中的 `import`，使其适配新路径

```
my_project/  
|-- main.py  
|-- utils/  
    |-- __init__.py  
    |-- data_utils.py
```



进阶迭代：递归与生成器

■ 什么是递归？

- 递归是一种编程技术：函数调用自身，以求解可分解为更小同类子问题的问题。
- 递归函数必须包含两部分：
 - **基准情形：终止递归的简单条件。没有它会无限递归（并触发 RecursionError）。**
 - **递归步骤：函数调用自身的部分，但参数应更接近基准情形。**
- 类比：俄罗斯套娃。每个娃娃内含更小的自己，直到最小、无法打开的那个（基准情形）。

■ 经典示例：阶乘

The recursive definition of factorial $n!$:

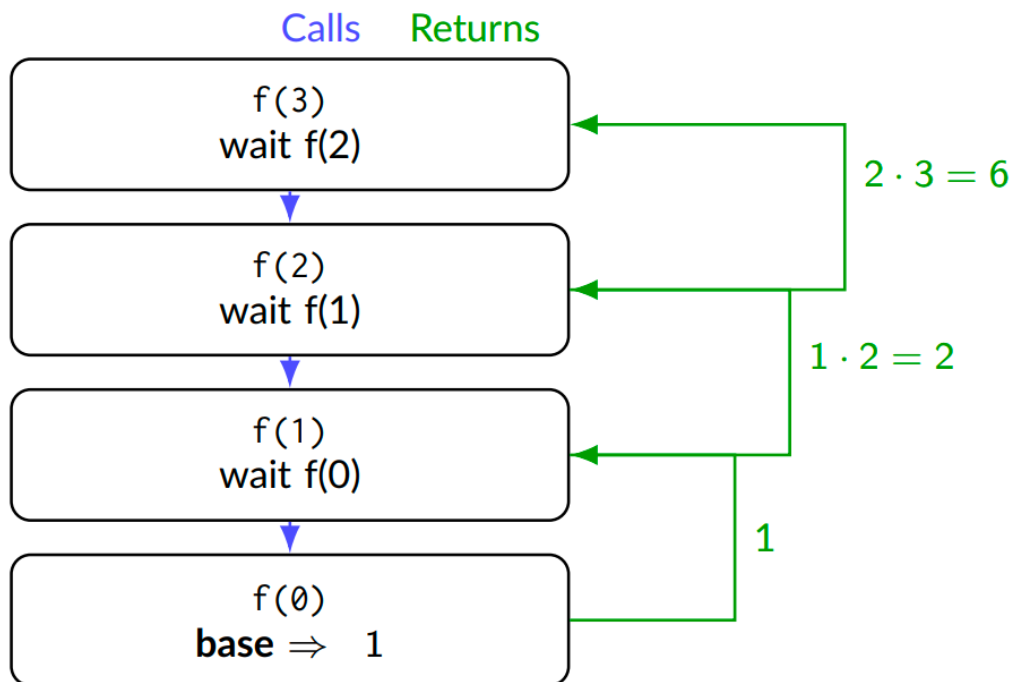
$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ (Base Case)} \\ n \times (n - 1)! & \text{if } n > 0 \text{ (Recursive Step)} \end{cases}$$

```
1 def factorial(n):
2     # Base Case:
3     if n == 0:
4         return 1
5     # Recursive Step:
6     else:
7         return n * factorial(n - 1)
8
9 print(factorial(4)) # 24
```

可视化递归：调用栈

How factorial(3) runs:

expansion then contraction



Compact Trace

- 1) $f(0) \Rightarrow 1$
- 2) $f(1) = 1 \cdot f(0) \Rightarrow 1$
- 3) $f(2) = 2 \cdot f(1) \Rightarrow 2$
- 4) $f(3) = 3 \cdot f(2) \Rightarrow 6$

Mental Model

- Expansion: push frames until base case.
- Contraction: pop frames, combine results.
- Each frame has its own n ; they don't interfere.

Checkpoints

- Base case reachable? (stops recursion)
- Progress toward base? ($n-1$)
- Work on return path? (e.g., $n * f(n-1)$)

递归：优缺点

优点

- **适合递归问题：**树、图、解析等天然递归任务可用简洁可读的方式表达。
- **简化思考：**将问题拆成自相似片段，避免复杂循环。

缺点

- **更多内存与时间：**每次调用新增栈帧，可能比迭代更慢。
- **深度限制风险：**过深递归可能触及 Python 调用栈上限（`RecursionError`）。

原则：代码更清晰时用递归；过深或简单循环优先用迭代。

生成器与 yield 关键字

生成器是返回惰性迭代器的特殊函数。
它按需逐个产生元素。

- 生成器函数使用 yield 而非 return。
- 遇到 yield 时，函数状态被“冻结”，向调用者送出值并等待。
- 当下一个值被请求时（如 for 循环），函数从暂停处继续执行

```
1 def count_up_to(max_val):
2     print("Generator starting...")
3     count = 1
4     while count <= max_val:
5         yield count # Pauses here and sends 'count' back
6         count += 1
7     print("Generator finished.")
8 # A for loop automatically handles the iteration
9 for number in count_up_to(3):
10    print(f"Received {number}")
```

动机：为何使用生成器？

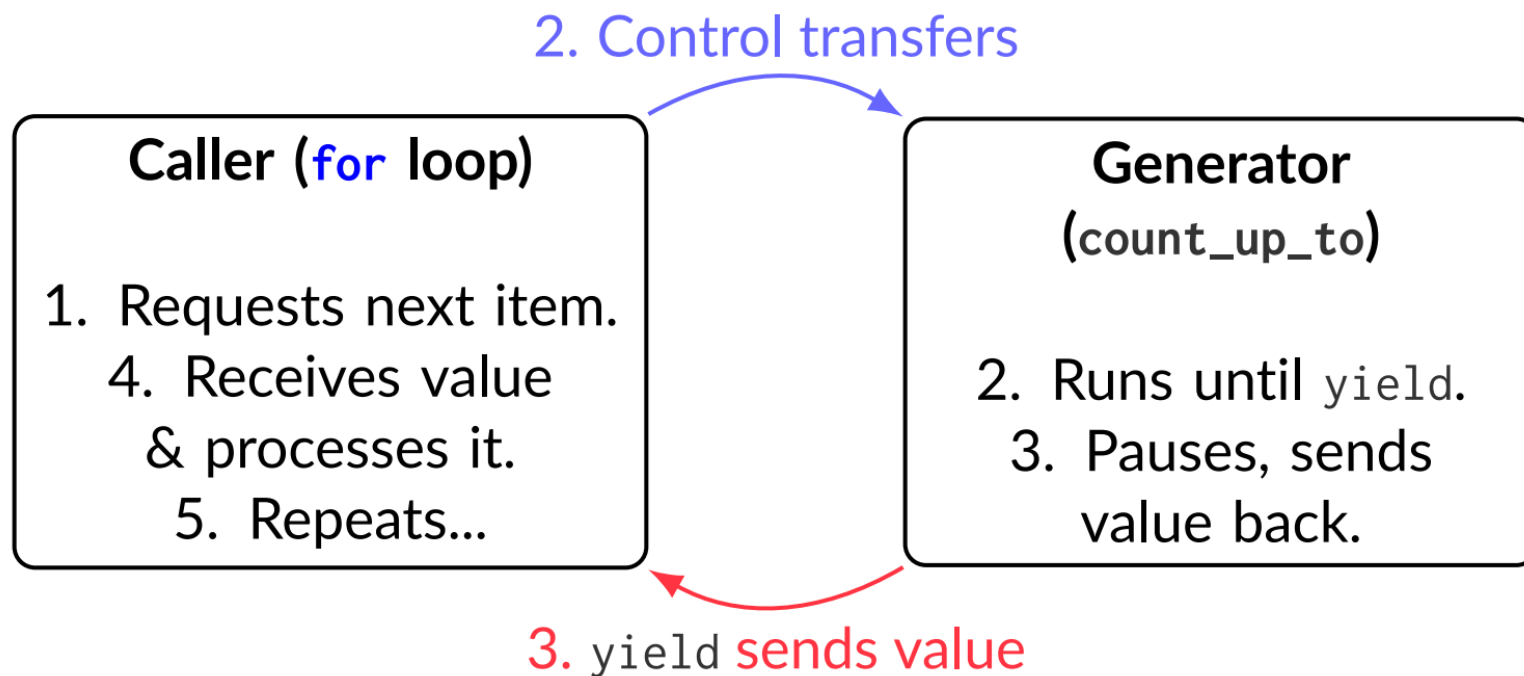
- 大数据集（如 50 GB 日志、数百万张图像）无法一次载入。急切读取会耗尽内存。
- 生成器实现惰性、省内存处理：一次产生一项，保持 RAM 占用很小。

```
1 # Avoid eager loading of huge files
2 def read_huge_file(path):
3     with open(path, 'r') as f:
4         return f.readlines() # Loads all lines into RAM
5                               -- risky
6
7 all_data = read_huge_file('large_dataset.csv') # Likely
8 to crash
```

可视化生成器：yield 之舞

- 该循环持续到生成器结束，内部抛出 `StopIteration`，从而干净地结束 `for` 循环。

How a `for` loop interacts with a generator:



从递归到生成器

递归适用之时

- 树/图遍历；分治中深度优先结构自然的场景。

其局限之处

- 对海量数据做长线性扫描：栈深度与反复函数调用会增加内存与执行时间。

引入生成器

- 惰性流式迭代：按需产生元素，额外内存为 $O(1)$ 。
- 组合流水线（map/filter），无需构建完整列表。

经验法则

- 递归形态 $\Rightarrow$ 递归
- 海量流式数据 $\Rightarrow$ 生成器

AI 应用：数据批次生成器

- 深度学习中，我们按批次训练模型，而非一次使用整个数据集。
- 生成器正适合这一节省内存的任务。
- 该方法一次生成一个批次，无需预先准备全部批次。

```
1 import numpy as np
2 def batch_generator(features, labels, batch_size):
3     """Yields batches of data for training."""
4     num_samples = len(features)
5     # Loop forever to allow for many epochs of training
6     while True:
7         indices = np.random.permutation(num_samples)
8         for i in range(0, num_samples, batch_size):
9             batch_indices = indices[i : i + batch_size]
10            yield features[batch_indices], labels[batch_indices]
11 # Usage with a large dataset:
12 # features, labels = load_my_huge_dataset()
13 # train_gen = batch_generator(features, labels, batch_size=32)
14 # for i in range(num_training_steps):
15 #     batch_x, batch_y = next(train_gen)
16 #     model.train_on_batch(batch_x, batch_y)
```

任务：用递归与生成器完成常见数据任务，并比较各自优势。

- 1. 递归搜索：编写递归函数 `find_files(fs_dict, ext)`，在嵌套字典中找出所有以给定扩展名 `ext` 结尾的文件路径**
- 2. 省内存日志扫描：编写生成器函数 `get_errors(log_path)`，仅 `yield` 以 "ERROR" 开头的行，适用于可能很大的日志文件，保持低内存占用。**
- 3. 比较方法：**
 - 说明：为何递归天然适合任务 1？
 - 说明：为何递归不适合任务 2，尤其面对数百万条日志时？

类与对象

- 面向对象编程、封装
- 构造/析构函数，类/对象级属性
- 魔术方法（__lt__、__str__、__len__、__getitem__）

继承

- 子类/父类、多态

模块与包

- 内置模块、自定义模块、import
- __name__=='__main__', 包

进阶迭代：递归与生成器

- yield 语句

人工智能 Python 编程

感谢聆听!